

1 背景与动机

随着区块链系统持续运行，账户状态、合约状态以及由 Merkle Patricia Trie (MPT) 组织的中间节点会不断累积。若每个全节点都完整复制全部状态数据，系统可以获得较直接的数据可用性保证，但也会带来很高的存储开销，并限制长期可扩展性。为了降低单个节点的存储负担，一种自然思路是将原始 MPT node 静态切分并交给不同节点保存。然而，一次 key 查询需要恢复从 root 到目标叶子的完整路径；当路径节点被分散后，请求方不仅需要掌握各原始节点的位置，还会依赖多个指定持有者及时在线。在开放的公有链网络中，节点加入、退出和短时离线频繁发生，这种对确定位置和确定副本的依赖容易引入尾延迟、超时重试和局部失效。

传统复制机制虽然简单，但为了抵抗节点失效通常需要较高的冗余度，因而会抵消原始数据切分带来的存储节省。纠删码能够在理论上以更低冗余提供恢复能力，但固定分块编码通常要求请求方明确知道每个缺失块的位置，并依赖对应持有者返回特定块。在公有链场景中，消息主要依靠局部 gossip 传播，请求方不能依赖某个指定节点稳定完成响应，也不适合在每次读取中通过多轮协商逐一确认缺失块。因此，编码后的任意有效响应应当能够被累积使用，而不是只有某个预定块才能推进恢复过程。

本文面对的核心问题可以概括为：在一个大规模、动态、局部可观测的公有链网络中，如何在减少全量状态复制的同时，仍然保证被划分和裁剪后的状态路径可以被恢复并验证。这里的数据恢复不仅要求能够收集足够多的有效片段，还要求接收方能够确认这些片段确实来自声明的状态节点及线性关系，否则恶意或错误的编码结果会破坏后续状态重构。由于本文关注的是状态可用性与恢复正确性，而不是通过全局协调消除所有节点失效，因此系统设计必须尽量依赖本地可获得的信息，并将全局一致性要求压缩到少量公开、可重建的参数上。

基于上述背景，本文采用面向 MPT 路径的编码存储方案。网络中的参与者均为地位对等的存储节点，不再区分专门的索引节点、编码节点或恢复节点。所有节点保留当前 epoch 末高度的完整 MPT；对于同一 epoch 内更早的状态版本，系统不再重复保存完整历史路径，而是保留发生变化的历史叶子、版本跳表以及预先生成的喷泉编码符号。对于每个已经最终确认的 root 和 key，节点按照统一规则从对应访问路径计算一个 commitment，并以 key 作为数据分配和查询的共同入口。

为了支持读取，请求方只需广播 key 与目标版本，不依赖额外的全局目

录或显式编码对象标识。节点根据公开的 key 分配规则判断请求是否命中本地数据，命中的节点并行返回预先保存的线性组合、seed 和有效性证明。请求方使用该 $(root, key)$ 对应的 commitment 验证 IPA 关系，只累积能够提升系数矩阵秩的有效组合，并在达到恢复门限后解码所需的 MPT 路径或历史叶子。该设计把喷泉码从临时传输手段变为实际持久化格式，使数据划分、节点离线容忍和多源恢复由同一套基于 key 的规则统一描述。

在验证层面，本文沿用 Pedersen 向量承诺和 IPA 证明验证编码块的线性关系，并可使用 folding 聚合同一 witness 下的多条内积关系。发送方除编码结果外还提供系数与路径关系，使验证方能够检查 $c = \langle a, b \rangle$ ；解码后的 MPT 节点仍需通过 node hash 和目标 root 验证。

2 数据验证

2.1 IPA 初始化参数与透明 CRS

本文采用透明公共参考串 (Transparent CRS) 初始化 IPA，而不依赖可信设置或需要事后销毁的秘密参数。设 $\mathbb{G}$ 为阶为大素数 q 的循环群， $\mathbb{F}_q$ 为对应的标量域。系统固定公开的群参数

$$pp_{\text{grp}} = (\mathbb{G}, q, \text{HashToGroup}, \text{DST}),$$

其中 HashToGroup 为标准化的哈希到群算法，DST 为本协议专用的域分离标签。群及哈希算法一经协议确定便保持不变，任何节点均不得使用本地随机数替换这些参数。

系统以 epoch 为参数更新周期。设第 e 个 epoch 包含 L 个区块，其中 L 可根据底层链的最终确认窗口取 30 至 64 之间的固定值。待该 epoch 达到最终确认后，根据其共识状态导出

$$s_e = H(\text{chainID} \parallel e \parallel R_e \parallel D_e),$$

其中 R_e 为由已最终确认区块头、VRF 输出或 RANDAO 混合值确定的 epoch 随机性， D_e 为该 epoch 的难度统计量。这里的 s_e 只依赖全网一致的最终确认状态，不依赖单个节点观测到的局部出块时间。为避免当前 epoch 尚未结束造成参数不确定， s_e 与 n_{e+1} 用于初始化第 $e+1$ 个 epoch。

给定下一 epoch 的向量维度 n_{e+1} ，其 IPA 公共基底按索引确定性生成：

$$g_i = \text{HashToGroup}(\text{DST} \parallel s_e \parallel \mathbb{G} \parallel i), \quad h_i = \text{HashToGroup}(\text{DST} \parallel s_e \parallel \mathbb{H} \parallel i),$$

$$u = \text{HashToGroup}(\text{DST} \parallel s_e \parallel \mathbf{U}), \quad i = 1, \dots, n_{e+1}.$$

因此, 第 $e+1$ 个 epoch 的 IPA 参数为

$$\text{CRS}_{e+1} = (\{g_i\}_{i=1}^{n_{e+1}}, \{h_i\}_{i=1}^{n_{e+1}}, u).$$

不同标签 $\mathbf{G}$ 、 $\mathbf{H}$ 与 $\mathbf{U}$ 用于保证各类基点之间的域分离。实现时还应执行群成员与子群检查, 并在哈希结果为单位元或出现重复基点时按公开规则重新派生。由于全部输入均公开且确定, 证明方和验证方只需知道 `chainID`、epoch 编号、最终确认的 R_e, D_e 以及 n_{e+1} , 即可独立重建同一组基底。

透明 CRS 生成的是 IPA 与 Pedersen 向量承诺使用的公共代数基底, 而不是喷泉码每条关系中的系数 α_i 。后者仍由编码规则或 Fiat-Shamir transcript 产生。系统必须满足如下维度一致性条件:

$$\dim(\mathbf{b}) = \dim(\mathbf{a}_i) = n_{e+1} = |\{g_i\}| = |\{h_i\}|.$$

若实际向量长度不足 n_{e+1} , 则按照协议规定的规范顺序补零; 超过该维度时则分批构造 statement, 不允许证明方临时截断或扩展 CRS。这样, epoch 给出稳定的时间边界, 难度决定 witness 的划分维度, 透明 CRS 则给出与该维度严格一致的代数承诺空间。

2.2 单行喷泉码 IPA 验证

首先, 我们对数据 (节点或区块) 向量 $\mathbf{b} = (b_1, b_2, \dots, b_n)$ 进行 Pedersen 形式的承诺

$$Q = \mathbf{Com}(\mathbf{b}) = \prod_{i=1}^n g_i^{b_i}$$

在喷泉码广播过程中, 生成的编码块为

$$c = \alpha_1 b_1 + \alpha_2 b_2 + \dots + \alpha_n b_n$$

这可以写成向量 α 与 b 的内积。因此, 我们可以利用 IPA 来证明: 由系数向量 α 生成的编码块确实为 c 。传统 IPA 的承诺形式如下:

$$P = \left(\prod_{i=1}^n h_i^{\alpha_i} \right) \left(\prod_{i=1}^n g_i^{b_i} \right) u^{\langle \alpha, b \rangle}$$

但是, 由于 P 本身并非公开可直接计算的量, 验证方无法据此确认其中的 b 就是先前承诺于 Q 的向量。因此, 我们将 P 改写为一个能够由公开信息

和承诺值共同确定的公开量。

$$P = \left(\prod_{i=1}^n h_i^{\alpha_i} \right) \cdot Q \cdot u^{\langle \alpha, b \rangle}$$

由于除 b 之外，其余信息均为公开信息，因此验证方可以结合 Q 计算出 P ，再根据 proof 的验证结果来确认内积关系是否成立，从而判断所传输的编码块是否确实由系数向量 α 生成

2.3 多行喷泉码 IPA 验证

在我们的场景中，如果对每一条关系都分别生成一份 IPA 证明，那么证明数量会随着关系条数线性增长，从而带来较大的证明与管理开销。

为此，我们引入 *folding*。其作用并不是改变单条 IPA 的验证规则，而是将多条同类型的内积关系递归地压缩为一个新的根关系，使系统最终只需对该根 statement 生成一次 IPA 证明。这样，证明入口便可从 n statements 统一为 one root statement。

$$x_i = (Q, a_i, c_i), \quad c_i = \langle a_i, b \rangle.$$

其中 $Q = \text{Com}(b)$ 表示对秘密向量 b 的承诺，而 a_i 与 c_i 为公开量，现考虑两条叶子 statement

$$(Q, a_1, c_1), \quad (Q, a_2, c_2).$$

给定一个公开挑战 χ ，我们定义折叠后的公开向量与公开结果为

$$a^* = a_1 + \chi a_2, \quad c^* = c_1 + \chi c_2.$$

由于内积在公开侧满足线性，有

$$\langle a^*, b \rangle = \langle a_1 + \chi a_2, b \rangle = \langle a_1, b \rangle + \chi \langle a_2, b \rangle = c_1 + \chi c_2 = c^*.$$

因此，折叠后的父节点 statement 仍然保持同样的形式，即

$$x^* = (Q, a^*, c^*).$$

这里不需要让 b 参与 folding，是因为所有叶子共享的是同一个秘密 witness b 。折叠过程仅作用于公开约束一侧，而秘密侧及其承诺 Q 在整个

过程中保持不变。通过递归地重复上述折叠规则，多条叶子关系可以逐层聚合为唯一的根 statement

$$x_{\text{root}} = (Q, a_{\text{root}}, c_{\text{root}}),$$

其中 a_{root} 与 c_{root} 均由所有叶子在公开挑战序列下递归组合得到。此后，证明方仅需针对该根 statement 生成一次普通的 IPA 证明，即证明

$$\exists b: Q = \text{Com}(b) \wedge \langle a_{\text{root}}, b \rangle = c_{\text{root}}.$$

对于验证方而言，由于全部叶子 statement 以及全部挑战值均为公开信息，因此其可以自行重建整棵 folding tree，并得到相同的根 statement。最终，验证过程只需完成两个检查：其一，确认根 statement 是否由叶子按照 folding 规则正确导出；其二，确认该根 statement 的 IPA proof 是否通过。只要这两项同时成立，验证方即可推出所有原始叶子关系均成立。

2.4 基于 Folding 的 IPA 证明与验证流程

我们考虑如下场景：验证方需要验证 n 条叶子关系

$$\langle a_i, b \rangle = c_i, \quad i = 1, \dots, n,$$

其中， a_i 为公开向量， b 为秘密 witness，并且

$$Q = \text{Com}(b)$$

是对 b 的公开承诺。

2.4.1 证明方准备

对于每个叶子，证明方构造对应的 statement：

$$x_i = (a_i, c_i, Q).$$

该叶子 statement 的含义是：同一个秘密 witness b 满足

$$\langle a_i, b \rangle = c_i.$$

2.4.2 折叠生成根

证明方按照公开的 folding 规则和公开 challenge, 对叶子 statement 进行递归两两折叠。对于两个子 statement

$$(a_1, c_1, Q), \quad (a_2, c_2, Q),$$

给定公开 challenge χ , 定义折叠后的父节点为

$$a^* = a_1 + \chi a_2, \quad c^* = c_1 + \chi c_2.$$

由于所有叶子共享同一个秘密 witness b , 因此秘密侧的承诺保持不变:

$$Q^* = Q.$$

通过重复上述折叠过程, 所有叶子最终被聚合为一个根 statement:

$$x_{\text{root}} = (a_{\text{root}}, c_{\text{root}}, Q).$$

随后, 证明方根据根 statement 计算对应的根 IPA 承诺:

$$P_{\text{root}} = \left(\prod_{i=1}^n h_i^{a_i^{\text{root}}} \right) \cdot \left(\prod_{i=1}^n g_i^{b_i} \right) \cdot u^{c_{\text{root}}} = H(a_{\text{root}}) \cdot Q \cdot u^{c_{\text{root}}}$$

其中, $H(a_{\text{root}})$ 表示公开向量部分对应的承诺项, Q 绑定秘密 witness b , 而 $u^{c_{\text{root}}}$ 绑定公开的内积结果。

在整个 folding 过程中, 只有公开侧的 a_i 和 c_i 被递归组合; 秘密向量 b 不参与折叠, 其承诺 Q 在所有层级中保持不变。

2.4.3 生成根 IPA Proof

折叠完成后, 根 statement 可以被视为一条普通的 IPA statement:

$$\langle a_{\text{root}}, b \rangle = c_{\text{root}}.$$

证明方基于根承诺 P_{root} 和秘密 witness b , 生成一份根 IPA proof:

$$\pi_{\text{root}} = \text{IPA.Prove}(P_{\text{root}}, a_{\text{root}}, c_{\text{root}}; b).$$

该过程与普通 IPA proof 的生成方式一致: 证明方以 P_{root} 作为公共承诺输入, 递归划分向量并生成每一轮的 L/R 元素, 最终得到 proof transcript。证明方随后将证明 π_{root} 以及叶子数据 $\{(a_i, c_i)\}_{i=1}^n$ 发送给验证方。

2.4.4 验证方重建与验证

验证方收到叶子数据后, 使用相同的公开 challenge 和 folding 规则, 独立重建 folding 过程, 从而得到

$$a_{\text{root}}^{(\text{local})}, \quad c_{\text{root}}^{(\text{local})}.$$

然后, 验证方根据本地重建出的根 statement 计算根 IPA 承诺:

$$P_{\text{root}}^{(\text{local})} = H\left(a_{\text{root}}^{(\text{local})}\right) \cdot Q \cdot u_{c_{\text{root}}^{(\text{local})}}.$$

由于本文采用单边 witness 结构, 秘密向量 b 及其承诺 Q 在折叠过程中始终不变, 内积结果只需随公开向量 a_i 按相同系数线性组合, 不会引入额外的交叉项。最后, 验证方使用该重建的根承诺验证证明方提供的根 IPA proof:

$$\text{IPA.Verify}\left(P_{\text{root}}^{(\text{local})}, \pi_{\text{root}}\right) \stackrel{?}{=} 1.$$

如果验证通过, 则说明根 relation 成立。由于该根 statement 是由验证方固定的叶子数据按照公开 folding 规则重建得到的, 因此验证方可以接受所有叶子关系

$$\langle a_i, b \rangle = c_i, \quad i = 1, \dots, n$$

均成立。若验证失败, 则说明根 proof 无效, 或者至少有一条叶子关系与承诺的 witness 不一致。

folding 的作用是将 n 条叶子约束压缩为一个根约束, 从而避免为每个叶子单独生成 IPA proof。承诺 Q 在整个过程中保持不变, 用于绑定秘密 witness b 。证明方只对折叠后的根 statement 生成一次 IPA proof, 而验证方可以根据公开叶子数据独立重建相同的根承诺, 并使用该根承诺验证 proof。

因此, 该流程将原本需要 n 份 IPA proof 的验证任务, 转化为一次根 IPA proof 的验证。证明目标仍然是所有叶子关系同时成立, 但证明方式变为先递归折叠为根, 再对根统一证明。

3 数据划分

本方案以 root 和 key 唯一确定的 MPT 访问路径作为数据组织、承诺和编码的基本单位。网络中的全部参与者均为对等存储节点, 它们共同保留当前 epoch 末高度的完整 MPT, 并按照公开的 key 分配规则分散保存历史

叶子及其预生成喷泉编码符号。数据划分因此不是将原始 MPT node 互斥地切入若干 logical partition, 而是将同一 $(root, key)$ 对应的不同线性组合分配给多个节点。请求方读取状态时, 根据 key 触达潜在持有者, 并从并行响应中收集足够多的、通过验证且线性无关的组合, 以恢复目标路径或历史叶子。

3.1 基于 Key 的节点分配与承诺生成

系统以状态 key 作为数据划分和节点定位的基本输入。第 e 个 epoch 的全部对等存储节点形成公开的 DHT 覆盖网络, 每个节点根据 epoch 随机数和自身公钥得到确定的 DHT 标识。对于 key k , 系统计算

$$x_k = H(\text{chainID} \| k),$$

并按照 DHT 距离选择距离 x_k 最近的一组有效节点, 分别保存与 k 有关的不同喷泉编码符号; 当某个节点达到存储容量上限时, 分配沿 DHT 顺序继续扩展到后续节点。这里的“负责范围”只表示节点在 DHT 上应当保存和响应哪些 key, 不形成独立的节点类型。任意节点都使用相同规则参与存储、响应和验证, 并可以仅根据 key 计算查询和数据分配应当到达的范围。

DHT 划分只由 key 决定, 因此同一个 key 在当前 epoch 内产生的多个历史版本仍然落在相同的逻辑负责范围内。高度或 version 不参与 DHT 的基础位置计算, 而是用于区分该 key 在不同状态版本下的编码内容。这使负责节点能够围绕同一个 key 连续保存其最新 path 和历史叶子编码, 同时避免为每个高度重新建立一套节点分配关系。

对于每一个已经最终确认的高度 h , 以 r_h 统一表示该高度的 root。key k 在该 root 下唯一确定一条 MPT 访问路径

$$\text{Path}(r_h, k) = (r_h, v_1, \dots, v_\ell),$$

后续节点按照 k 的 nibble 序列依次连接。协议规定统一的节点序列化、路径顺序、字段映射和补零规则。所有完成该高度状态执行的节点都针对每个 (r_h, k) 独立计算且仅计算一个 Pedersen 承诺

$$Q_{r_h, k} = \text{Commit}(\text{Path}(r_h, k)).$$

root、root node 与 stateRoot 在本文中均指这里的 r_h , 不再分别引入其他根标识。由于承诺输入和公共参数均由最终确认状态确定, 全部对等节点都可以在本地得到相同的 $Q_{r_h, k}$, 不依赖单独的承诺生成者或额外的承诺目录。

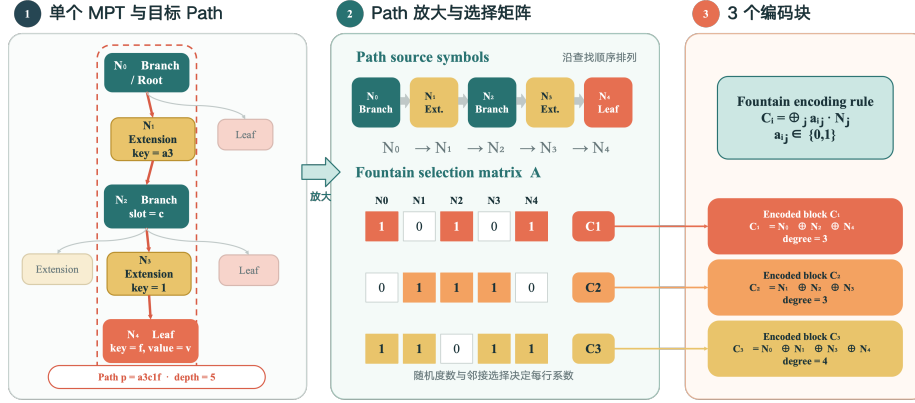


图 1: 基于 Key 的编码操作

因此, key 同时连接了节点分配与路径承诺: DHT 根据 k 确定哪些节点保存和响应相应数据, 目标版本先确定 r_h , 再由 (r_h, k) 唯一确定 path 及其 $Q_{r_h, k}$ 。编码、查询和验证均直接使用这组信息, 不再维护额外的编码对象标识。

3.2 喷泉编码与持久化对象

对于 (r_h, k) 对应的路径, 节点先按照规范序列化规则得到源向量 $\mathbf{b}_{r_h, k}$, 再根据公开的符号种子 $\text{seed}_{r_h, k, j}$ 生成系数向量

$$\mathbf{a}_{r_h, k, j} = \text{CoefficientVector}(\text{seed}_{r_h, k, j}, K_{r_h, k}),$$

并计算第 j 个喷泉编码符号

$$c_{r_h, k, j} = \langle \mathbf{a}_{r_h, k, j}, \mathbf{b}_{r_h, k} \rangle = \sum_{t=1}^{K_{r_h, k}} a_{r_h, k, j, t} b_{r_h, k, t}.$$

编码使用与承诺和 IPA 兼容的有限域, 使同一线性关系既能用于喷泉恢复, 也能直接作为 IPA 的内积 statement。编码器针对该关系生成证明

$$\pi_{r_h, k, j} = \text{IPA.Prove}(Q_{r_h, k}, \mathbf{a}_{r_h, k, j}, c_{r_h, k, j}).$$

系统实际分配和持久化的对象为

$$Z_{r_h, k, j} = (r_h, k, \text{seed}_{r_h, k, j}, c_{r_h, k, j}, \pi_{r_h, k, j}).$$

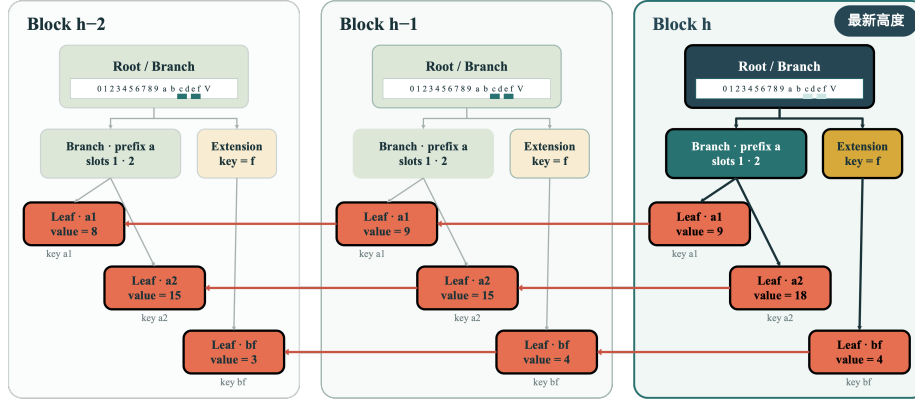


图 2: 多版本 MPT 历史状态数据存储

系数向量不需要完整存储，可以由 seed 和公开的向量长度重新生成。每个对等节点按照 DHT 分配结果持久化一部分 $Z_{r_h,k,j}$ ，无需额外保存编码上下文标识。全部节点继续保留当前 epoch 末高度的完整 MPT；对于同一 epoch 内已经被后续状态替代的历史路径，当足够数量的编码符号完成分发并通过存储确认后，只保留历史叶子、跳表关系和被分配的编码对象，不再重复保存其完整父节点路径。

设 (r_h, k) 对应的数据向量长度为 $K_{r_h,k}$ ，协议共生成 $N_{r_h,k}$ 个编码符号，其中

$$N_{r_h,k} > K_{r_h,k}.$$

冗余符号用于吸收节点离线、网络超时、符号重复以及部分符号无效造成的损失。请求方不要求收到某些指定符号，只要获得足够多的、通过验证且系数向量线性无关的符号，使恢复方程达到满秩，就可以恢复 $\mathbf{b}_{r_h,k}$ 。

3.3 Epoch 内多版本叶子压缩与跳表索引

设一个 epoch 内包含多个已经最终确认的 MPT 版本。全部对等存储节点完整保留该 epoch 最后一个高度的 MPT；对于同一 epoch 中更早的版本，不再保存完整的历史父节点和访问路径，只保留各 key 曾经出现过的历史叶子。相同叶子在连续高度内保持有效时不重复记录，只有叶子发生变化时才增加一个新的历史版本。

每个历史叶子只记录它开始加入当前版本序列的高度，并通过哈希指针连接到更早的叶子。除指向前一个版本的基本指针外，部分记录还保存跨

越多历史版本的跳跃指针，从而形成从最新叶子向旧叶子回溯的认证跳表。epoch 末完整 MPT 中的最新叶子作为该 key 的跳表入口；该叶子已经包含在对应的 $Q_{r_h,k}$ 中，其认证哈希指针继续绑定后续历史链接。历史记录不需要显式保存结束高度，因为下一条更新记录的高度已经自然给出了前一版本的结束位置。

查询 key k 在目标高度 h 下的状态时，请求方从最新叶子开始沿跳跃指针向前查找，并选择加入高度不大于 h 的最近一条叶子记录。该记录就是高度 h 对应的有效版本。由于记录按照高度有序，且最新叶子由 $Q_{r_h,k}$ 承诺、后续记录由哈希指针逐级认证，请求方可以同时确认叶子版本的先后关系，而不需要为每个历史叶子额外保存一个显式高度范围。

在 epoch 数据编码阶段，最新 MPT 中的一条完整 path 与该 key 的多个历史叶子共同参与喷泉码线性组合。若最新 path 记为 A ，历史叶子记为 $B_1, B_2, \dots$ ，则一个预生成编码关系可写为

$$c_j = \langle \mathbf{a}_j, A \rangle + \sum_t \beta_{j,t} B_t.$$

这些历史叶子不再携带各自原路径中的父节点。请求方根据 key 和目标高度确定对应的 root 与编码数据，再从多个对等节点收集相关线性组合，恢复最新 path 和所需的历史叶子。

历史叶子的正确性仍由其生成高度下对应的 $(root, key)$ commitment 保证。编码时，历史叶子作为其原 path 的叶子位置参与线性关系；验证时，节点按照数据验证章节的 IPA 规则使用相应 commitment 检查该关系。每个 $(root, key)$ 始终只对应一个 commitment，验证过程也不要求继续保存已经删除的历史父节点。

通过这种方式，一个 epoch 的多版本状态被压缩为一棵末高度完整 MPT 和若干挂接在最新叶子之后的历史叶子跳表。MPT 负责最新状态索引，跳表负责版本定位，预生成喷泉码负责分散保存最新 path 与历史叶子。

3.4 数据读取、传输与恢复验证

一次数据读取由 key 驱动。请求方构造

$$q = (k, h, \text{requestID}),$$

并通过 DHT 或邻居 gossip 将请求广播到网络。请求中只需要给出目标 key、目标高度和请求编号，不需要提前指定某个存储节点，也不需要携带独立的

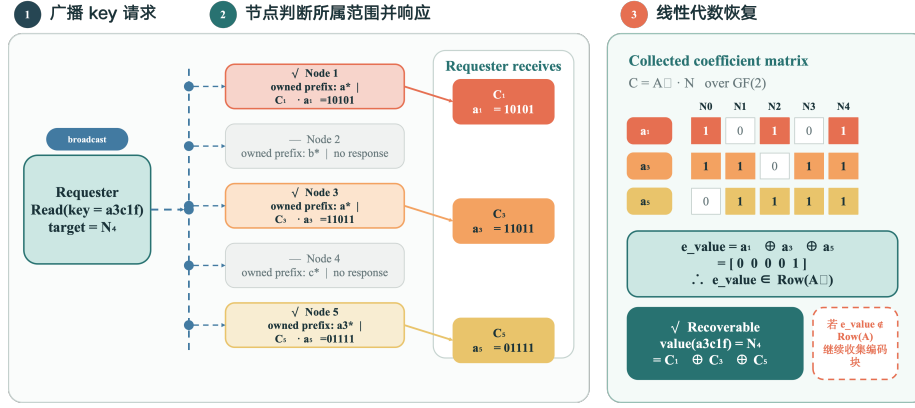


图 3: 状态数据远程读取

编码对象标识。高度 h 唯一确定 root r_h , 因而 (r_h, k) 已经能够唯一确定本次请求对应的数据和 commitment。

网络中的节点收到 q 后, 根据 key 计算其 DHT 标识, 并在本地判断该 key 是否落入自己的负责范围。节点还会检查本地是否实际保存了 (r_h, k) 对应的编码符号。若该 key 不属于自己的责任范围, 或者本地没有相关数据, 节点保持静默; 只有命中的节点参与响应。由此, 查询可以采用广播方式发出, 而数据筛选由各节点根据公开的 key 分配规则独立完成。

参与响应的节点不需要重新读取完整 MPT path, 也不在收到请求后临时执行喷泉编码。节点直接从本地取出预先生成并持久化的线性组合

$$c_j = \langle a_j, b_{r_h, k} \rangle,$$

并返回

$$(\text{requestID}, \text{seed}_j, c_j, \pi_j).$$

其中 seed_j 用于重建公开系数向量 a_j , π_j 用于证明 c_j 确实由对应承诺下的数据向量生成。不同节点保存不同 seed 对应的编码符号, 因此它们可以针对同一个请求并行发送不同的线性关系。

请求方收到响应后, 首先根据 seed 重建系数向量, 再使用 $Q_{r_h, k}$ 验证 IPA proof。未通过证明、请求编号不一致、系数重复或不能提高当前系数矩阵秩的响应被直接丢弃。通过验证的编码符号被加入恢复方程组:

$$Ab = c.$$

请求方不依赖某个指定节点完整返回全部数据，只要从任意在线节点获得足够多的有效且线性无关的组合，使 A 达到恢复所需的满秩条件，就可以求解原始数据向量。

解码完成后，请求方按照规范格式还原其中的 MPT path、叶子或多版本数据，并使用对应的 $(root, key)$ commitment、节点哈希关系和目标 root 检查恢复结果。对于包含历史叶子的聚合向量，历史叶子的版本关系同时由其跳表指针和加入高度验证。只有编码关系、承诺关系、版本关系和 MPT 哈希关系全部成立时，读取结果才被接受。

例如，设高度 h 的 root 为 r_h ，key k_1 对应的路径为

$$\text{Path}(r_h, k_1) = \{r_h, x_1, x_2, x_3\}.$$

请求方广播 $q = (k_1, h, \text{requestID})$ 。节点 N_a, N_b, N_c 根据 k_1 的 DHT 标识判断该 key 属于自己的负责范围，并发现本地保存了对应编码符号，因此分别返回

$$c_a = \langle \mathbf{a}_a, \mathbf{b}_{r_h, k_1} \rangle, \quad c_b = \langle \mathbf{a}_b, \mathbf{b}_{r_h, k_1} \rangle, \quad c_c = \langle \mathbf{a}_c, \mathbf{b}_{r_h, k_1} \rangle$$

以及各自的 seed 和 IPA proof；其他未命中的节点不响应。请求方使用 Q_{r_h, k_1} 验证这些关系，只保留能够提升秩的符号。当有效方程达到满秩后，请求方恢复 $\{r_h, x_1, x_2, x_3\}$ ，再检查 $r_h \rightarrow x_1 \rightarrow x_2 \rightarrow x_3$ 的 child hash 关系。若目标高度对应跳表中的历史叶子，命中的节点返回包含该叶子的预编码关系，请求方再结合叶子加入高度、跳跃指针和对应的 $(root, key)$ commitment 完成同样的恢复与验证。

该流程将数据定位与数据恢复分离：key 广播负责触达潜在持有者，DHT 责任判断负责节点自筛选，预生成喷泉符号负责多节点并行传输，IPA 与满秩解码负责从不确定的响应集合中恢复可验证数据。节点离线、超时或返回重复符号不会改变读取对象，请求方只需继续累积其他节点返回的有效线性组合。